

Egison Core

Demand-Directed Typing の形式仕様

目次

1	目的と読み方	2
2	構文と判断	3
3	一回の checking cut	5
4	式の synthesis 規則	7
5	Pattern と matcher の規則	9
6	機械化された性質と未完成の接続	11

1 目的と読み方

1.1 この仕様が定めるもの

本稿は、Egison core の source program が型付け可能であることを定める形式仕様である。公開する source typing は DDTyping 一つだけであり、その内部を型の合成（synthesis）と検査（checking）に分ける。

$$\begin{aligned} q; S; \widehat{\Sigma}; \Gamma \vdash e \Rightarrow \tau_{\text{raw}} \dashv q'; S' & \quad \text{synthesis,} \\ q; S; \widehat{\Sigma}; \Gamma \vdash e \Leftarrow \rho \dashv q'; S' & \quad \text{checking.} \end{aligned}$$

$q = (q_\kappa, q_\tau)$ は二つの独立な自然数 counter の組である。 q_κ は次に生成する capability metavariable の番号、 q_τ は次に生成する target metavariable の番号を表す。例えば $q = (3, 5)$ なら、次の capability variable は χ_3 、次の target variable は α_5 である。一方だけを生成したときは、対応する counter だけを進める。 S はその位置までに得た capability / target の paired substitution である。どの規則も子を左から右へ調べ、出力 q', S' を次の子へ渡す。

閉じた wrapper は canonical initial supply q_0 と恒等置換から開始し、最後にだけ raw result を解決する。

$$\text{DDTyping}(\widehat{\Sigma}, \Gamma, e, \tau) \iff \exists \tau_{\text{raw}}, q', S'. \text{DDSynth}_{\widehat{\Sigma}}(q_0, \text{id}, \Gamma, e, \tau_{\text{raw}}, q', S') \wedge \tau = S' \tau_{\text{raw}}.$$

動的安全性の証明では、推論 state を消去した内部 certificate RuntimeTyping を用いる。これは runtime value typing と preservation の帰納法を簡潔にするための補助判断であり、source acceptance を定めない。本稿の source typing rule はすべて DDTyping family で記述する。目指す依存方向は

$$\text{DDTyping} \implies \text{RuntimeTyping} \implies \text{RuntimeSafety}$$

である。最初の矢印には capability freeze provenance の統合がまだ必要である。

1.2 中心となる考え：producer と slot demand

Matcher $\kappa\tau$ は matcher を生成する producer type, MatcherSlot $\kappa\tau$ は matcher を消費する位置の expected type である。checking は常に次の一手順を取る。

synthesize $e \longrightarrow$ その出力 cut で一度だけ expected type と align する
--

非恒等 coercion を選べるのは、その cut で $S\rho$ の head が MatcherSlot と判明している場合だけである。したがって coercion の場所は matchAll など特定の構文に固定されない。例えば

$$\text{use} : \text{MatcherSlot} \kappa\tau \rightarrow \rho, \quad m : \text{Matcher} \kappa\tau$$

の下で use m を型付けすると、関数適用規則が domain を先に確定し、引数 m の checking cut で producer と slot が対応付けられる。matchAll の matcher 引数, matcher clause の next-matcher, slot domain を持つ高階関数の引数は、すべて同じ DDCheck を使う。したがって「一度だけ」とは導出木全体で一度という意味ではなく、一つの checking cut ごとに一度という意味である。

resolved source と expected の組は次の表で分類する.

resolved source	resolved expected	alignment
Matcher $\kappa_p \tau_p$	MatcherSlot $\kappa_c \tau_c$	matcher-to-slot
product of matchers	MatcherSlot $\kappa_c \tau_c$	product lift; matcher-to-slot
product of slots	MatcherSlot $\kappa_c \tau_c$	slot-tuple lift
MatcherSlot $\kappa_p \tau_p$	MatcherSlot $\kappa_c \tau_c$	slot-to-slot equality
その他	その他	ordinary equality

expected が型変数のままなら ordinary equality だけを行う. coercion のために未解決変数を slot へ推測しない. また ordinary equality の失敗を見てから coercion を試す rollback も行わない. resolved head が branch を先に決め、選んだ solve が失敗すればその checking は失敗する.

1.3 章の案内

第2章は型・source form・judgment の記号を導入する. 第3章は本方式の核である一回の checking cut と alignment を定義する. 第4章は通常の式, 第5章は pattern と matcher literal の規則である. 第6章は機械化済みの性質と, まだ主張していない境界を分けてまとめる. 各 Lean module と回帰試験の詳細は../docs/details.md に譲る.

2 構文と判断

2.1 型とスキーム

capability κ と target type τ は別 sort である. matcher と slot は同じ二つの index を持つが, 前者は producer, 後者は consumer expectation を表す.

$\kappa ::= \text{Any} \mid \chi \mid \hat{s} \mid K \bar{\kappa} \mid (\kappa_1, \dots, \kappa_n)$	$\kappa : \text{Cap},$
$\tau ::= \alpha \mid \hat{a} \mid \mathbf{1} \mid \text{Int} \mid \text{Bool} \mid K \bar{\tau} \mid (\tau_1, \dots, \tau_n) \mid \tau_1 \rightarrow \tau_2$ $\mid \text{Matcher } \kappa \tau \mid \text{MatcherSlot } \kappa \tau$	$\tau : \text{Type},$
$\sigma ::= \forall(\bar{\chi} : \text{Cap}). \forall(\bar{\alpha} : \text{Type}). \tau$	expression scheme,
$\eta ::= \kappa \triangleright \tau$	pattern dual,
$\varsigma ::= \forall(\bar{\chi} : \text{Cap}). \forall(\bar{\alpha} : \text{Type}). (\bar{\eta} \Rightarrow \eta)$	pattern-function scheme.

χ, α は flexible metavariable, $\hat{s}, \hat{a}$ は rigid skolem である. Any は one-way capability matching の consumer 側でだけ wildcard として働き, 対称 unification では通常の rigid constructor である. $\text{mono } \tau$ は量化 binder を持たない scheme を表す.

2.2 ソース構文

$e ::= x \mid \lambda x. e \mid e_1 e_2 \mid \text{let } x = e_1 \text{ in } e_2 \mid \text{fix } f \text{ x.e} \mid n$ $\mid (e_1, \dots, e_n) \mid C \bar{e} \mid \text{op}(\bar{e}) \mid \text{something}$ $\mid \text{matcher } cls$ $\mid \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e,$	
$p ::= \$x \mid _ \mid \#e \mid \tilde{x} \mid c \bar{p} \mid p_1 \& p_2 \mid p_1 \mid p_2 \mid f \bar{p} \mid (\bar{p}),$	
$pp ::= \$ \mid _ \mid \# \$x \mid c \bar{pp} \mid (\bar{pp}),$	
$dp ::= \$x \mid _ \mid C \bar{dp} \mid (\bar{dp}),$	
$cl ::= pp \text{ as } M \text{ with } [dp_j \rightarrow N_j]_{j=1}^r,$	
$cls ::= [cl_i]_{i=1}^m.$	

c は pattern constructor, C は data constructor である. surface match は matchAll と head への elaboration で

あり, core primitive ではない. pattern-function declaration は source expression の typing より前に freeze され, canonical signature の一部になる.

2.3 シグネチャ・文脈・状態

$$\begin{aligned} \widehat{\Sigma} = (\widehat{\Sigma}_D, \widehat{\Sigma}_P, \widehat{\Sigma}_F, \widehat{\Sigma}_{prim}, Obs_{\widehat{\Sigma}}) & \quad \text{freeze 済み canonical signature,} \\ \Gamma ::= \emptyset \mid \Gamma, x : \sigma & \quad \text{expression scheme context,} \\ \Phi ::= \emptyset \mid \Phi, x : \eta & \quad \text{pattern-parameter context,} \\ \Delta ::= \emptyset \mid \Delta, x : \tau & \quad \text{monomorphic pattern bindings,} \\ q = (q_\kappa, q_\tau) & \quad \text{next capability / target variable numbers,} \\ S = (C, T) & \quad \text{prevailing paired substitution.} \end{aligned}$$

q_κ と q_τ は独立な自然数 counter であり、それぞれ次に未使用の capability metavariable と target metavariable の番号を保持する。以下では

$$\chi_{q_\kappa} : \text{Cap}, \quad \alpha_{q_\tau} : \text{Type}, \quad q + \langle i, j \rangle = (q_\kappa + i, q_\tau + j)$$

と書く。従って capability variable を一つ生成すれば supply は $q + \langle 1, 0 \rangle$, target variable なら $q + \langle 0, 1 \rangle$, 両方を一つずつ生成する pattern variable 規則では $q + \langle 1, 1 \rangle$ となる。番号空間は sort ごとに別なので、 χ_3 と α_3 は別の変数である。closed wrapper の初期 supply は, signature と context に既出の番号より各 counter が大きくなるように選ぶ。

S の作用は capability を先に, target substitution を後に適用する。

$$S\kappa = C\kappa, \quad S\tau = T(C\tau), \quad S\Gamma = \{x : S\sigma \mid x : \sigma \in \Gamma\}.$$

substitution delta は seq で時系列順に prevailing substitution へ合成する。後段の capability action は前段の target range 内部にも作用するため、二 component を独立に合成しない。

2.4 判断の一覧

$\text{DDSynth}_{\widehat{\Sigma}}(q, S, \Gamma, e, \tau, q', S')$	expression synthesis
$\text{DDCheck}_{\widehat{\Sigma}}(q, S, \Gamma, e, \rho, q', S')$	expression checking
$\text{DDSynths}_{\widehat{\Sigma}}, \text{DDChecks}_{\widehat{\Sigma}}$	expression-list traversal
$\text{DDPattern}_{\widehat{\Sigma}}(q, S, \Gamma, \Phi, \Delta, p, \eta, \Delta', q', S')$	user-pattern synthesis
$\text{DDPPat}_{\widehat{\Sigma}}(q, S, pp, \tau, \bar{\eta}, \Delta, q', S')$	primitive-pattern checking
$\text{DDDPat}_{\widehat{\Sigma}}(q, S, dp, \tau, \Delta, q', S')$	data-pattern checking
$\text{DDClause}_{\widehat{\Sigma}}(q, S, \Gamma, cl, \tau, \bar{\eta}, q', S')$	matcher-clause inference
$\text{DDArms}_{\widehat{\Sigma}}, \text{DDClauses}_{\widehat{\Sigma}}$	arm / clause-list traversal

list judgment は空列で q, S をそのまま返す。cons では head の出力を tail の入力にし, binding context も左から右へ延長する。lookup または規則中の部分関数が失敗した場合、その規則は適用できない。

2.5 具体化と一般化

Inst_q と InstDual_q は supply-indexed に量化 binder を fresh variable へ置き換え, 次 supply も返す。expression context と pattern-function signature の量化 capability binder は capability variable へだけ instantiate される。利用側の demand を根拠に producer capability を constructor や product へ変えることはできない。一方, data / pattern constructor と primitive の scheme は通常の binder-supported structural instance を許す。

Gen は signature と context に自由な variable を除いて量化する。

$$\text{Gen}(\widehat{\Sigma}, \Gamma, \tau) = \forall(\text{fcv}(\tau) \setminus (\text{fcv}(\widehat{\Sigma}) \cup \text{fcv}(\Gamma))). \forall(\text{ftv}(\tau) \setminus (\text{ftv}(\widehat{\Sigma}) \cup \text{ftv}(\Gamma))). \tau.$$

scheme binder は局所的であり、後段の substitution が導入した同番号の自由 variable を捕捉しない。let は value の synthesis 直後の $S\Gamma, S\tau$ を一般化する。

3 一回の checking cut

3.1 制約を解く関係

$\text{DDAlignTypes}(S, \tau, \rho, S')$ は coercion を伴わない equality alignment, $\text{DDAlign}(S, \tau, \rho, S')$ は checking cut 全体である。どちらも入力 S を破壊せず, constraint を解く局所 delta を S の後へ合成して S' を得る。

ExactCapMGU と ExactPairedMGU は, それぞれ capability と paired type constraint の proof-carrying MGU である。MGU の soundness と因子化に加え, constraint 外で恒等, range が constraint 内, solved form が冪等であることを要求する。この exactness により, unification と無関係な metavariable を同時に構造化する guess を除く。

$\text{OneWayDelta}(\kappa_p, \tau_p, \kappa_c, \tau_c, D)$ は producer を固定したまま consumer capability と target を解く exact delta である。consumer Any は wildcard, consumer variable は同じ raw occurrence 間で一貫した値を要求する。producer 側の capability を consumer demand に合わせて強化してはならない。

3.2 通常の等価整合

matcher 同士または slot 同士では capability を先に解き, その delta を target へ作用させてから target constraint を解く。それ以外の型は paired type 全体を一度に解く。

$$\frac{S\tau = \text{Matcher } \kappa_1 \tau_1 \quad S\rho = \text{Matcher } \kappa_2 \tau_2 \quad \text{ExactCapMGU}(\kappa_1, \kappa_2, C) \quad \text{ExactPairedMGU}(C\tau_1, C\tau_2, T)}{\text{DDAlignTypes}(S, \tau, \rho, \text{seq}(T, \text{seq}((C, \text{id}), S)))}$$

[ALIGN-MATCHER]

$$\frac{S\tau = \text{MatcherSlot } \kappa_1 \tau_1 \quad S\rho = \text{MatcherSlot } \kappa_2 \tau_2 \quad \text{ExactCapMGU}(\kappa_1, \kappa_2, C) \quad \text{ExactPairedMGU}(C\tau_1, C\tau_2, T)}{\text{DDAlignTypes}(S, \tau, \rho, \text{seq}(T, \text{seq}((C, \text{id}), S)))}$$

[ALIGN-SLOT]

$$\frac{\text{alignPairClass}(S\tau, S\rho) = \text{ordinary} \quad \text{ExactPairedMGU}(S\tau, S\rho, D)}{\text{DDAlignTypes}(S, \tau, \rho, \text{seq}(D, S))}$$

[ALIGN-ORDINARY-TYPES]

3.3 Demand classifier と coercion の選択

$\text{demandClass}(S\tau, S\rho)$ は resolved type だけから次の五つの class を決める。product of matchers, product of slots の順に調べるため, 空 product は $\text{productMatcherLift}$ が優先される。

$\text{productMatcherLift}, \text{slotTupleLift}, \text{matcherToSlot}, \text{slotToSlot}, \text{ordinary}.$

$$\frac{\text{productMatcherDuals?}(S\tau) = \text{some } \bar{\eta} \quad S\rho = \text{MatcherSlot } \kappa v \quad \text{OneWayDelta}((\bar{\eta}, \bar{\kappa}), (\bar{\eta}, \bar{\tau}), \kappa, v, D)}{\text{DDAlign}(S, \tau, \rho, \text{seq}(D, S))}$$

[ALIGN-PRODUCT-MATCHER]

$$\frac{\text{demandClass}(S\tau, S\rho) = \text{slotTupleLift} \quad \text{productSlotDuals?}(S\tau) = \text{some } \bar{\eta} \quad S\rho = \text{MatcherSlot } \kappa v \quad \text{ExactCapMGU}((\bar{\eta}, \bar{\kappa}), \kappa, C) \quad \text{ExactPairedMGU}(C(\bar{\eta}, \bar{\tau}), C v, T)}{\text{DDAlign}(S, \tau, \rho, \text{seq}(T, \text{seq}((C, \text{id}), S)))}$$

[ALIGN-SLOT-TUPLE]

$$\frac{S\tau = \text{Matcher } \kappa_p \tau_p \quad S\rho = \text{MatcherSlot } \kappa_c \tau_c \quad \text{OneWayDelta}(\kappa_p, \tau_p, \kappa_c, \tau_c, D)}{\text{DDAlign}(S, \tau, \rho, \text{seq}(D, S))}$$

[ALIGN-MATCHER-SLOT]

$$\frac{S\tau = \text{MatcherSlot } \kappa_1 \tau_1 \quad S\rho = \text{MatcherSlot } \kappa_2 \tau_2 \quad \text{ExactCapMGU}(\kappa_1, \kappa_2, C) \quad \text{ExactPairedMGU}(C\tau_1, C\tau_2, T)}{\text{DDAlign}(S, \tau, \rho, \text{seq}(T, \text{seq}((C, \text{id}), S)))}$$

[ALIGN-SLOT-SLOT]

$$\frac{\text{demandClass}(S\tau, S\rho) = \text{ordinary} \quad \text{DDAlignTypes}(S, \tau, \rho, S')}{\text{DDAlign}(S, \tau, \rho, S')}$$

[ALIGN-ORDINARY]

product-of-matchers branch は, whole-product matcher の生成と matcher-to-slot 消費という二 primitive step を一つの checking cut で行う. 別の構文位置でもう一度 coercion する規則ではない. non-ordinary class はすべて resolved expected head が slot の場合だけである. matcher-headed expected type は必ず ordinary alignment へ退化する.

3.4 唯一の checking 規則

式の checking 規則は次の一つだけである. expected type は synthesis の入力にならず, synthesis が返したその cut の S_1 でだけ alignment を選ぶ.

$$\frac{q; S; \Gamma \vdash e \Rightarrow \tau \dashv q_1; S_1 \quad \text{DDAlign}(S_1, \tau, \rho, S')}{q; S; \Gamma \vdash e \Leftarrow \rho \dashv q_1; S'} \quad [\text{DD-CHECK}]$$

4 式の synthesis 規則

以下では $q; S; \Gamma \vdash e \Rightarrow \tau \dashv q'; S'$ を $\text{DDSynth}_{\Sigma}(q, S, \Gamma, e, \tau, q', S')$ の略記とする. α_{q_τ} は supply q が指す次の target variable である.

4.1 変数・関数・適用

変数 lookup は prevailing substitution を context に適用してから scheme を fresh instantiate する. lambda domain は任意に選ばず, 必ず fresh metavariable とする. 関数適用は function を先に function type へ align し, その後で argument を domain に対して check する.

$$\begin{array}{c} \frac{(S\Gamma)(x) = \sigma \quad \text{Inst}_q(\sigma) = (\tau, q')}{q; S; \Gamma \vdash x \Rightarrow \tau \dashv q'; S} \quad [\text{DD-VAR}] \qquad \frac{q + \langle 0, 1 \rangle; S; \Gamma, x : \text{mono } \alpha_{q_\tau} \vdash e \Rightarrow \tau \dashv q'; S'}{q; S; \Gamma \vdash \lambda x. e \Rightarrow \alpha_{q_\tau} \rightarrow \tau \dashv q'; S'} \quad [\text{DD-LAM}] \\[2ex] \frac{\begin{array}{c} q; S; \Gamma \vdash e_1 \Rightarrow \tau_f \dashv q_1; S_1 \\ \text{DDAlignTypes}(S_1, \tau_f, \alpha_{(q_1)_\tau} \rightarrow \alpha_{(q_1)_{\tau+1}}, S_2) \quad q_1 + \langle 0, 2 \rangle; S_2; \Gamma \vdash e_2 \Leftarrow \alpha_{(q_1)_\tau} \dashv q_2; S_3 \end{array}}{q; S; \Gamma \vdash e_1 e_2 \Rightarrow \alpha_{(q_1)_\tau+1} \dashv q_2; S_3} \quad [\text{DD-APP}] \end{array}$$

この順序により, function domain が slot に解決された場合, argument の構文に関係なくその一つの DDCheck cut で matcher-to-slot coercion を選べる.

4.2 リテラル・タプル・コンストラクタ

$$\begin{array}{c}
\frac{}{q; S; \Gamma \vdash n \Rightarrow \text{Int} \dashv q; S} \\
\text{[DD-LIT]}
\end{array}
\qquad
\frac{\text{DDSynths}_{\widehat{\Sigma}}(q, S, \Gamma, \bar{e}, \bar{\tau}, q', S')}{q; S; \Gamma \vdash (\bar{e}) \Rightarrow (\bar{\tau}) \dashv q'; S'} \\
\text{[DD-TUPLE]}$$

$$\frac{\text{Inst}_q(\widehat{\Sigma}_D(C)) = (\bar{\rho} \rightarrow \rho, q_0) \quad \text{DDChecks}_{\widehat{\Sigma}}(q_0, S, \Gamma, \bar{e}, \bar{\rho}, q', S')}{q; S; \Gamma \vdash C \bar{e} \Rightarrow \rho \dashv q'; S'} \\
\text{[DD-CON]}$$

$$\frac{\text{Inst}_q(\widehat{\Sigma}_{\text{prim}}(\text{op})) = (\bar{\rho} \rightarrow \rho, q_0) \quad \text{DDChecks}_{\widehat{\Sigma}}(q_0, S, \Gamma, \bar{e}, \bar{\rho}, q', S')}{q; S; \Gamma \vdash \text{op}(\bar{e}) \Rightarrow \rho \dashv q'; S'} \\
\text{[DD-PRIM]}$$

DDSynths と DDChecks は head の出力 state を tail へ渡すため, constructor と primitive の引数も左から右へ処理される.

4.3 Let と再帰

$$\frac{q; S; \Gamma \vdash e_1 \Rightarrow \tau_1 \dashv q_1; S_1 \quad \sigma = \text{Gen}(\widehat{\Sigma}, S_1 \Gamma, S_1 \tau_1) \quad q_1; S_1; \Gamma, x : \sigma \vdash e_2 \Rightarrow \tau_2 \dashv q'; S'}{q; S; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 \Rightarrow \tau_2 \dashv q'; S'} \\
\text{[DD-LET]}$$

$$\frac{f \neq x \quad \text{DirectSelf}(f, e) \quad \text{NonMatcherBody}(e) \quad q + \langle 0, 2 \rangle; S; \Gamma, f : \text{mono}(\alpha_{q_\tau} \rightarrow \alpha_{q_\tau+1}), x : \text{mono } \alpha_{q_\tau} \vdash e \Rightarrow \rho \dashv q_1; S_1 \quad \text{DDAlignTypes}(S_1, \rho, \alpha_{q_\tau+1}, S')}{q; S; \Gamma \vdash \text{fix } f \ x.e \Rightarrow \alpha_{q_\tau} \rightarrow \alpha_{q_\tau+1} \dashv q_1; S'} \\
\text{[DD-FIX]}$$

$\text{DirectSelf}(f, e)$ は shadowing-aware な構文条件である. f の自由出現を application spine の head に限り, bare value, 引数位置, alias 経由, 高階の受渡しを拒否する. matcher literal を body に持つ recursion は第 5 章の専用 placeholder 規則を使う.

4.4 標準 matcher producer

$$\frac{}{q; S; \Gamma \vdash \text{something} \Rightarrow \text{Matcher Any } \alpha_{q_\tau} \dashv q + \langle 0, 1 \rangle; S} \\
\text{[DD-SOMETHING]}$$

something 自身は matcher producer である. slot が必要かどうかはこの規則では決めず, 外側の DDCheck cut が expected head を見て決める.

5 Pattern と matcher の規則

5.1 ユーザーパターンの synthesis

DDPattern は pattern dual $\eta = \kappa \triangleright \tau$ と binding context を同時に合成する. Δ は左から右へ延長し, 同じ branch 内の重複 binder を拒否する.

$$\frac{x \notin \text{dom}(\Delta)}{\text{DDPattern}_{\widehat{\Sigma}}(q, S, \Gamma, \Phi, \Delta, \$x, \chi_{q\kappa} \triangleright \alpha_{q\tau}, \Delta, x : \alpha_{q\tau}, q + \langle 1, 1 \rangle, S)} \quad [\text{DD-PAT-VAR}]$$

$$\frac{}{\text{DDPattern}_{\widehat{\Sigma}}(q, S, \Gamma, \Phi, \Delta, _, \chi_{q\kappa} \triangleright \alpha_{q\tau}, \Delta, q + \langle 1, 1 \rangle, S)} \quad [\text{DD-PAT-WILD}]$$

$$\frac{q; S; \text{mono } \Delta, \Gamma \vdash e \Rightarrow \tau \dashv q_1; S_1}{\text{DDPattern}_{\widehat{\Sigma}}(q, S, \Gamma, \Phi, \Delta, \#e, \chi_{(q_1)\kappa} \triangleright \tau, \Delta, q_1 + \langle 1, 0 \rangle, S_1)} \quad [\text{DD-PAT-VALUE}]$$

$$\frac{\Phi(x) = \kappa \triangleright \tau}{\text{DDPattern}_{\widehat{\Sigma}}(q, S, \Gamma, \Phi, \Delta, \widetilde{x}, \kappa \triangleright \tau, \Delta, q, S)} \quad [\text{DD-PAT-EMBED}]$$

tuple と constructor は子 pattern を先に合成する. constructor は instantiated field target と子 target を align し, 観測可能位置から result capability を決め, 最後に CapCompatible を検査する.

$$\frac{\text{DDPatterns}_{\widehat{\Sigma}}(q, S, \Gamma, \Phi, \Delta, \bar{p}, \bar{\eta}, \Delta', q', S')}{\text{DDPattern}_{\widehat{\Sigma}}(q, S, \Gamma, \Phi, \Delta, (\bar{p}), (\bar{\eta}.\bar{\kappa}) \triangleright (\bar{\eta}.\bar{\tau}), \Delta', q', S')} \quad [\text{DD-PAT-TUPLE}]$$

$$\frac{\text{Inst}_q(\widehat{\Sigma}_P(c)) = (\bar{p} \Rightarrow_P \rho, q_0) \quad \text{DDPatterns}_{\widehat{\Sigma}}(q_0, S, \Gamma, \Phi, \Delta, \bar{p}, \bar{\eta}, \Delta', q_1, S_1) \quad \text{DDAlignTargetList}(S_1, \bar{\eta}, \bar{p}, S_2) \quad \text{DDPatternCtorCap}_{\widehat{\Sigma}}(c, q_1, S_2, \bar{\eta}.\bar{\kappa}, \kappa, q_2, S_3) \quad \text{CapCompatible}_{\widehat{\Sigma}}(c; S_3 \bar{\eta}.\bar{\kappa}, S_3 \kappa)}{\text{DDPattern}_{\widehat{\Sigma}}(q, S, \Gamma, \Phi, \Delta, c \bar{p}, \kappa \triangleright \rho, \Delta', q_2, S_3)} \quad [\text{DD-PAT-CON}]$$

$p_1 \& p_2$ は左の bindings を右へ渡す. $p_1 | p_2$ は同じ入力 Δ から両 alternative を調べ, dual と binder 名・binder type を最後に align する.

$$\frac{\text{DDPattern}(q, S, \Gamma, \Phi, \Delta, p_1, \eta_1, \Delta_1, q_1, S_1) \quad \text{DDPattern}(q_1, S_1, \Gamma, \Phi, \Delta_1, p_2, \eta_2, \Delta_2, q_2, S_2) \quad \text{DDAlignDual}(S_2, \eta_1, \eta_2, S')}{\text{DDPattern}(q, S, \Gamma, \Phi, \Delta, p_1 \& p_2, \eta_1, \Delta_2, q_2, S')} \quad [\text{DD-PAT-AND}]$$

$$\frac{\text{DDPattern}(q, S, \Gamma, \Phi, \Delta, p_1, \eta_1, \Delta_1, q_1, S_1) \quad \text{DDPattern}(q_1, S_1, \Gamma, \Phi, \Delta, p_2, \eta_2, \Delta_2, q_2, S_2) \quad \text{DDAlignDual}(S_2, \eta_1, \eta_2, S_3) \quad \text{DDAlignBindings}(S_3, \Delta_1, \Delta_2, S')}{\text{DDPattern}(q, S, \Gamma, \Phi, \Delta, p_1 | p_2, \eta_1, \Delta_1, q_2, S')} \quad [\text{DD-PAT-OR}]$$

pattern-function scheme は freeze 済み signature から instantiate する. declaration の検査は source expression の DDTyping の外側であり, 本稿に declaration typing rule は置かない.

$$\frac{\text{InstDual}_q(\widehat{\Sigma}_F(f)) = (\bar{\eta} \Rightarrow \eta, q_0) \quad \text{DDPatterns}(q_0, S, \Gamma, \Phi, \Delta, \bar{p}, \bar{\eta}', \Delta', q_1, S_1) \quad \text{DDAlignDualList}(S_1, \bar{\eta}', \bar{\eta}, S')}{\text{DDPattern}(q, S, \Gamma, \Phi, \Delta, f \bar{p}, \eta, \Delta', q_1, S')} \quad [\text{DD-PAT-APP}]$$

5.2 MatchAll の型付け

`matchAll` は `target` を合成し, `pattern target` と `align` した後, `matcher expression` を `slot type` に対して check する. 第四 premise は, DD-MATCHALL 自身が直接導入する唯一の `slot demand` である. ただし, これは導出木全体に `demand` が一つしかないという意味ではない. 例えば e_m が `matcher literal` なら, その `synthesis` 中に各 `clause` の `next matcher` がそれぞれ別の `slot` に対して check される. 各 `checking cut` が独立に, 高々一度の `coercion` を行う.

$$\frac{q; S; \Gamma \vdash e_t \Rightarrow \tau_t \dashv q_1; S_1 \quad \text{DDPattern}(q_1, S_1, \Gamma, \emptyset, \emptyset, p, \kappa_p \triangleright \tau_p, \Delta, q_2, S_2) \quad \text{DDAlignTypes}(S_2, \tau_p, \tau_t, S_3) \quad q_2; S_3; \Gamma \vdash e_m \Leftarrow \text{MatcherSlot } \kappa_p \tau_t \dashv q_3; S_4 \quad q_3; S_4; \text{mono } \Delta, \Gamma \vdash e \Rightarrow \tau_r \dashv q'; S'}{q; S; \Gamma \vdash \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e \Rightarrow \text{List } \tau_r \dashv q'; S'}$$

[DD-MATCHALL]

5.3 Primitive pattern と data pattern の検査

DDPPat は一つの `shared matcher target` に対して `primitive pattern` を check し, `hole dual 列` と `bindings` を返す. `hole` だけが `fresh consumer capability` を作る.

$$\frac{}{\text{DDPPat}_{\widehat{\Sigma}}(q, S, \$, \tau, [\chi_{q_\kappa} \triangleright \tau], \emptyset, q + \langle 1, 0 \rangle, S)} \quad \text{[DD-PP-HOLE]}$$

$$\frac{}{\text{DDPPat}_{\widehat{\Sigma}}(q, S, _, \tau, [], \emptyset, q, S)} \quad \text{[DD-PP-WILD]}$$

$$\frac{}{\text{DDPPat}_{\widehat{\Sigma}}(q, S, \# \$ x, \tau, [], x : \tau, q, S)} \quad \text{[DD-PP-VALUE]}$$

`constructor` と `tuple` は `result` を `expected target` に `align` してから子へ `expected target 列` を渡す. DDPPats は `hole 列` を連結し, `binder` の `disjointness` を検査する.

$$\frac{\text{Inst}_q(\widehat{\Sigma}_P(c)) = (\bar{\rho} \Rightarrow_P \rho, q_0) \quad \text{DDAlignTypes}(S, \rho, \tau, S_1) \quad \text{DDPPats}_{\widehat{\Sigma}}(q_0, S_1, \bar{p}\bar{p}, \bar{\rho}, \bar{\eta}, \Delta, q', S')}{\text{DDPPat}_{\widehat{\Sigma}}(q, S, c \bar{p}\bar{p}, \tau, \bar{\eta}, \Delta, q', S')} \quad \text{[DD-PP-CON]}$$

$$\frac{\text{freshTargets}(n, q) = (\bar{\alpha}, q_0) \quad \text{DDAlignTypes}(S, (\bar{\alpha}), \tau, S_1) \quad \text{DDPPats}_{\widehat{\Sigma}}(q_0, S_1, \bar{p}\bar{p}, \bar{\alpha}, \bar{\eta}, \Delta, q', S')}{\text{DDPPat}_{\widehat{\Sigma}}(q, S, (\bar{p}\bar{p}), \tau, \bar{\eta}, \Delta, q', S')} \quad \text{[DD-PP-TUPLE]}$$

DDDPat は `arm` の `data pattern` を `clause target` に対して check し, `bindings` を返す.

$$\frac{}{\text{DDDPat}_{\widehat{\Sigma}}(q, S, \$ x, \tau, x : \tau, q, S)} \quad \text{[DD-DP-VAR]}$$

$$\frac{}{\text{DDDPat}_{\widehat{\Sigma}}(q, S, _, \tau, \emptyset, q, S)} \quad \text{[DD-DP-WILD]}$$

$$\frac{\text{Inst}_q(\widehat{\Sigma}_D(C)) = (\bar{\rho} \rightarrow \rho, q_0) \quad \text{DDAlignTypes}(S, \rho, \tau, S_1) \quad \text{DDDPats}_{\widehat{\Sigma}}(q_0, S_1, \bar{d}\bar{p}, \bar{\rho}, \Delta, q', S')}{\text{DDDPat}_{\widehat{\Sigma}}(q, S, C \bar{d}\bar{p}, \tau, \Delta, q', S')} \quad \text{[DD-DP-CON]}$$

$$\frac{\text{freshTargets}(n, q) = (\bar{\alpha}, q_0) \quad \text{DDAlignTypes}(S, (\bar{\alpha}), \tau, S_1) \quad \text{DDDPats}_{\widehat{\Sigma}}(q_0, S_1, \bar{d}\bar{p}, \bar{\alpha}, \Delta, q', S')}{\text{DDDPat}_{\widehat{\Sigma}}(q, S, (\bar{d}\bar{p}), \tau, \Delta, q', S')} \quad \text{[DD-DP-TUPLE]}$$

5.4 Clause と matcher literal の型付け

一つの clause は primitive pattern, next-matcher 列, arm 列の順に処理する. $\text{decomposeME}(M, n)$ は next expression を hole 数と同じ長さの expression 列へ展開する. 各 next matcher は対応する hole slot に対して DDCheck される.

$$\frac{\text{DDPPat}_{\widehat{\Sigma}}(q, S, pp, \tau, \bar{\eta}, \Delta_{pp}, q_1, S_1) \quad \text{decomposeME}(M, |\bar{\eta}|) = \text{some } \bar{M} \quad \text{DDChecks}_{\widehat{\Sigma}}(q_1, S_1, \Gamma, \bar{M}, \text{MatcherSlot } \eta.\kappa \eta.\tau, q_2, S_2) \quad \text{DDArms}_{\widehat{\Sigma}}(q_2, S_2, \Gamma, \Delta_{pp}, \bar{a}, \tau, \text{List prod}(\bar{\eta}.\tau), q', S')}{\text{DDClause}_{\widehat{\Sigma}}(q, S, \Gamma, pp \text{ as } M \text{ with } \bar{a}, \tau, \bar{\eta}, q', S')} \quad [\text{DD-CLAUSE}]$$

DDArms は data-pattern bindings と primitive-pattern bindings の衝突を拒否し, arm body を $\text{List prod}(\bar{\eta}.\tau)$ に対して check する.

matcher literal は全 clause で一つの fresh target を共有する. 最後の substitution で hole dual を一度だけ解決して evidence を再計算し, shape, catch-all, binder, arm exhaustiveness, coverage の全 checker を通した場合だけ matcher type を返す.

$$\frac{\text{DDClauses}_{\widehat{\Sigma}}(q + \langle 0, 1 \rangle, S, \Gamma, cls, \alpha_{q_\tau}, \bar{\eta}, q', S') \quad \text{collectClauseEvidence}_{\widehat{\Sigma}}(cls, \text{terminalHoleCaps}(S', \bar{\eta})) = \text{some } \bar{e}\bar{v} \quad \text{inferShape}_{\widehat{\Sigma}}(\bar{e}\bar{v}) = \text{some } \kappa \quad \text{clauseCapsListCheck}_{\widehat{\Sigma}}(\kappa, cls, \text{terminalHoleCaps}(S', \bar{\eta})) \quad \text{CatchAllLast}(cls) \quad \text{MatcherBindersCheck}(cls) \quad \text{ArmExhaustive}_{\widehat{\Sigma}_D}(cls, S' \alpha_{q_\tau}) \quad \text{CoverageOK}_{\widehat{\Sigma}}(cls, \kappa)}{q; S; \Gamma \vdash \text{matcher } cls \Rightarrow \text{Matcher } \kappa \alpha_{q_\tau} \dashv q'; S'} \quad [\text{DD-MATCHER}]$$

CatchAllLast は bare-hole catch-all を最後の clause に限定する. ArmExhaustive は保守的で実行可能な checker であり, core では variable または wildcard arm があれば成功する. CoverageOK は observable structured capability ごとに必要な constructor clause があることを検査する. checker の失敗を warning に落とす mode は core に含めない. matcher literal を body に持つ recursion は clause skeleton から domain / codomain placeholder を決める専用規則を使う.

$$\frac{f \neq x \quad \text{DirectSelf}(f, \text{matcher } cls) \quad \text{fixMatcherPlaceholderSupply}_{\widehat{\Sigma}}(cls, q) = \text{some } (\tau_d, \tau_c, q_0) \quad q_0; S; \Gamma, f : \text{mono } (\tau_d \rightarrow \tau_c), x : \text{mono } \tau_d \vdash \text{matcher } cls \Rightarrow \rho \dashv q_1; S_1 \quad \text{DDAlignTypes}(S_1, \rho, \tau_c, S')}{q; S; \Gamma \vdash \text{fix } f x.(\text{matcher } cls) \Rightarrow \tau_d \rightarrow \tau_c \dashv q_1; S'} \quad [\text{DD-FIX-MATCHER}]$$

NonMatcherBody により通常の DD-FIX と排他的である.

6 機械化された性質と未完成の接続

本節では, 前節までの規則から得られる性質と, 実行可能推論・動的安全性への接続をまとめる. solver の個別補題, trace invariant, 回帰一覧は `../docs/details.md` に分離する.

6.1 DDTyping 自身の性質

Slot-demand

$\text{DDAlign}(S, \tau, \rho, S')$ が ordinary equality でないなら, resolved expected type は必ず slot-headed である.

$$\text{DDAlign}(S, \tau, \rho, S') \implies \text{DDAlignTypes}(S, \tau, \rho, S') \vee \exists \kappa, v. S\rho = \text{MatcherSlot } \kappa v.$$

特に $S\rho$ が matcher-headed なら, どの DDAlign derivation も DDAlignTypes へ退化する. これにより「coercion は slot demand を観測した checking cut で一度だけ」という原則が judgment の定理になる.

State extension

すべての DD family は supply を単調に進め, 出力 substitution を入力 substitution と局所 solve delta の chronological replay に因子化できる. 公開型, pattern dual, bindings, hole ledger に現れる flexible variable は出力 supply で有界である. 従って closed wrapper が公開する型も canonical initial supply を拡張する終端 supply で有界である.

No guess

exact MGU は constraint 外で恒等であり, constraint variable の像を constraint 内に保ち, solved form である. 従って fresh lambda domain や無関係な metavariable を slot / matcher に構造化して coercion を成立させる derivation は存在しない.

代表的な境界

or-pattern

$$\text{matchAll } 0 \text{ as something with } (\$x \mid \$x) \rightarrow x$$

と, hole slot を something が満たす delegating matcher literal は DDTyping derivation を持つ. 一方, matcher-expected 位置へ product of matchers を渡す項と, fresh monomorphic domain を共有して demand のない product lift を必要とする項は derivation を持たない.

6.2 実行可能推論から内部 certificate へ

公開 inference は停止する raw traversal と有限の fail-closed validator の合成である.

$$\text{infer}(\widehat{\Sigma}, \Gamma, e) = \text{bind}(\text{inferRaw}(\widehat{\Sigma}, \Gamma, e), \lambda r. \text{if } \text{wBridgeCheck}(\widehat{\Sigma}, r) \text{ then some } r \text{ else none}).$$

raw traversal は fresh allocation, solve delta, generalization, coverage evidence, capability-origin ledger を trace に残す. validator はその有限 trace から binder-local instance, alignment, matcher finalization, let-generalization, freeze event を再検査し, reconstruction certificate を構成する. 型付け derivation 自体を oracle として受け取らない. $S_r = r.\text{state}.\text{prevailing}$ とすると, 公開成功から次の state-erasure 定理を得る.

$$\frac{\text{infer}(\widehat{\Sigma}, \Gamma, e) = \text{some } r}{\text{RuntimeTyping}(\widehat{\Sigma}, S_r \Gamma, e, r.\text{resolvedTarget})}.$$

RuntimeTyping は source acceptance の規則ではなく, solver state を持たない内部 certificate である. この定理は caller-supplied bridge や InferenceInputWF を前提にしない.

6.3 動的安全性

freeze 済み signature の実行可能 checker が FrozenSigWF を確立する。その下で, pristine な typed environment における評価は RuntimeTyping certificate を ValueTy へ保存する。matching machine については次を証明する。

1. typed state の一段の全 successor は typed である；
2. 非終端 typed state は局所的な StepReady の下で一段進む；
3. 一段保存の反復により, 到達可能な全 state は typed である；
4. 空 stack に到達した成功 branch の substitution は source pattern が生成した binding context で型付く。

空 successor 列は正当な match failure であり stuck ではない。一般の program termination は主張しない。primitive-pattern 内の value pattern capture admissibility は depth-first・左から右の core-order checker から導出する。

6.4 DDTyping から安全性へ残る接続

最終的に公開したい安全性定理の構造は次である。

$$\text{DDTyping}(\widehat{\Sigma}, \emptyset, e, \tau) \implies \text{RuntimeTyping}(\widehat{\Sigma}, \emptyset, e, \tau) \implies \text{RuntimeSafe}(\widehat{\Sigma}, e, \tau).$$

第二の矢印は機械化済みである。第一の矢印は現行 DD family にはまだ成り立たない。原因は, DDSynth が fresh supply と prevailing substitution を持つ一方, producer capability の origin と freeze event を index に持たないことである。

具体的には, context の量化 matcher scheme から生成した fresh capability や, let generalization が束縛した capability を, 後続の ordinary solve が Any や構造化 capability へ解く DD derivation が存在する。しかし runtime certificate の value-flow instance は, producer capability binder の像を capability variable に限定する。従って現行 DD derivation からは必要な variable-only instance を復元できない。

解決方針は, producer capability の生成・instance・generalization・export freeze の provenance を DD family 自体へ統合することである。その情報を derivation に沿って消去すれば RuntimeTyping certificate を構成できる。certificate の存在を DD rule の premise に置く方法は循環するため採らない。

6.5 受理完全性

もう一つの未完成定理は, DD derivation を executable inference の成功へ移すことである。現行 executable selector は expected type には cut 時点の prevailing substitution を適用するが, product source の認識には raw type を使う箇所がある。DD rule は cut-resolved source で classification するため, 両者を一致させる trace factorization が必要である。最終目標は追加 premise のない次の定理である。

$$\text{DDTyping}(\widehat{\Sigma}, \emptyset, e, \tau) \implies \text{infer}(\widehat{\Sigma}, \emptyset, e) \neq \text{none}.$$

6.6 機械化の範囲

DDSynth / DDCheck と pattern・arm・clause family, alignment の slot-demand, state replay, supply boundedness, 公開 inference から RuntimeTyping への reconstruction, concrete dynamic safety は Lean 4 に機械化済みである。主定理と一般補題に sorry, admit, project-defined axiom はない。